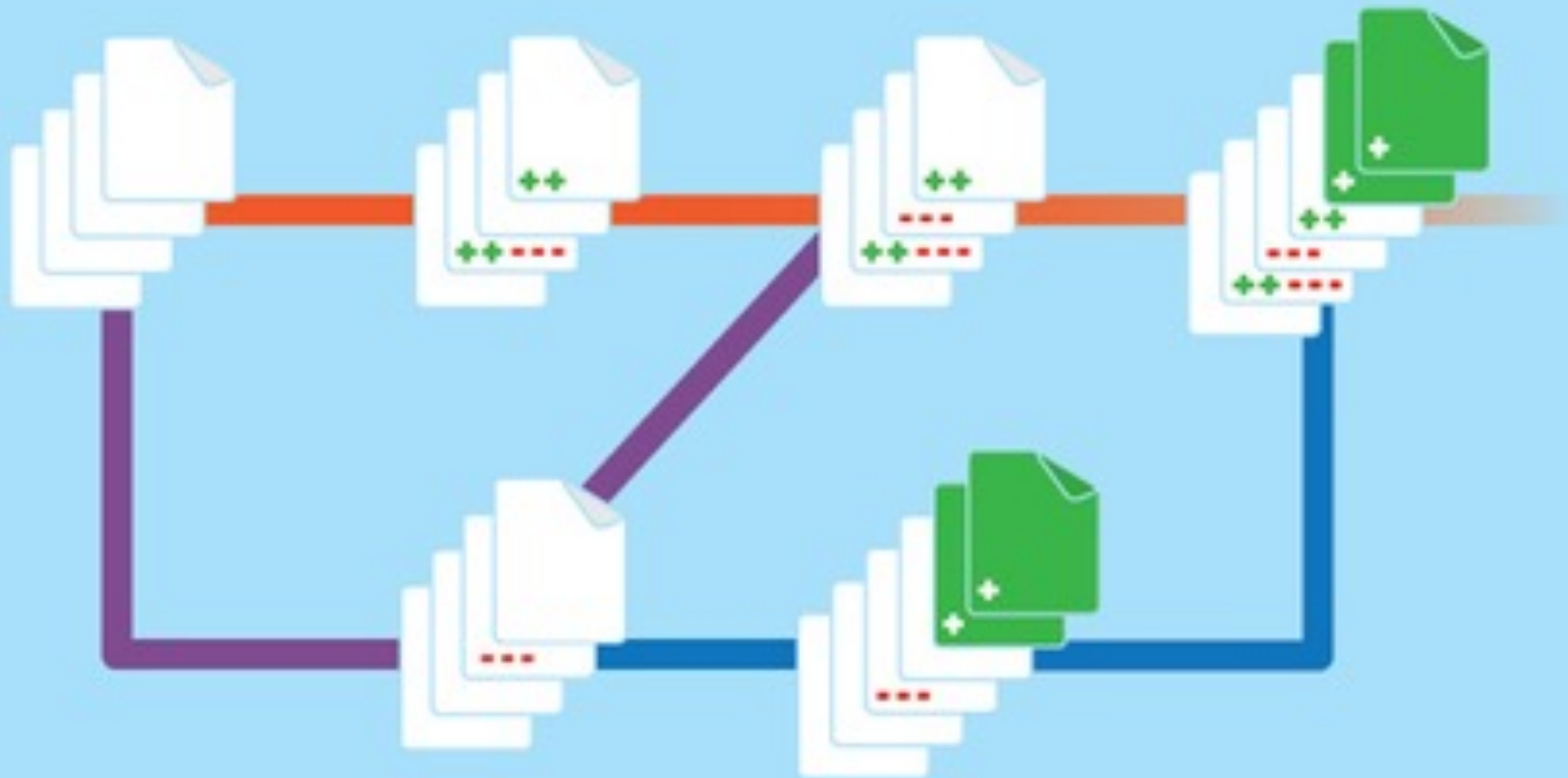




Gestão da Configuração e Versionamento

CÁSSIO CAPUCHO PEÇANHA

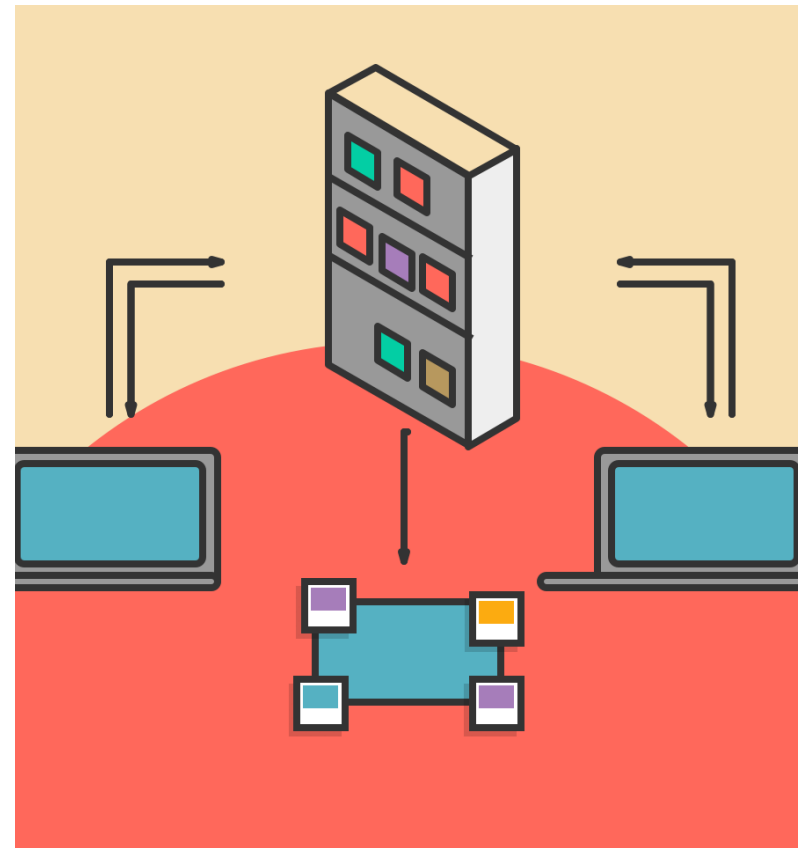


Controle de Versão

Como deixamos de apagar o código do colega e fomos para a nuvem.

○ Problema Original

- Software é desenvolvido em equipe.
- Precisamos de um local seguro para o código (o servidor).
- Precisamos de histórico (a máquina do tempo).
- O objetivo principal: poder desfazer operações (o famoso "CTRL+Z" do projeto inteiro)

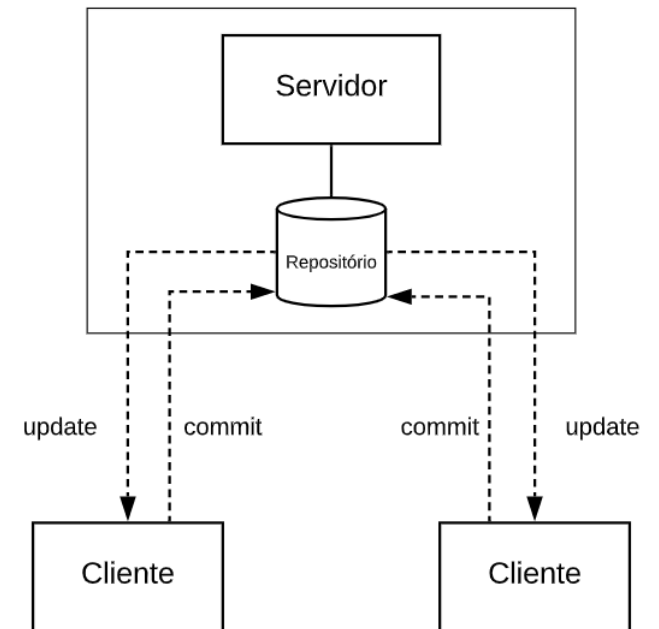


○ Problema Original

- Antes dos sistemas de versão, a equipe copiava a pasta inteira do projeto, compactava num .zip e colocava a data no nome (ex: `sistema_v1_01_03_1998.zip`).
- Por que isso parou de funcionar?
 - Se dois programadores trabalhassem ao mesmo tempo no mesmo .zip e tentassem juntar depois, eles tinham que ler linha por linha visualmente para saber o que mudou.
 - Com 2 pessoas já era um inferno. Com 5, o projeto travava porque ninguém sabia qual era o .zip oficial da semana.
- O controle local (zip) força a equipe a trabalhar de forma serial (um de cada vez), matando a produtividade.

A Era Centralizada (CVS e SVN)

- Os "Dinossauros" que organizaram a casa (anos 70 a 2000).
- Arquitetura Cliente/Servidor clássica. Existe **um único** servidor com o repositório.
- Para pegar a versão mais nova, você faz um *update*. Para enviar, um *commit*.
- "E se a internet cair ou o servidor queimar? O que acontece?"
 - Ninguém trabalha, pois o histórico fica preso no servidor.



A Era Centralizada (CVS e SVN)

Para evitar que um programador esmague o código do outro, sistemas como o SVN e o CVS trabalham com uma regra de ouro:

o servidor nunca resolve o conflito, ele devolve o problema para você.

A Era Centralizada (CVS e SVN)

O Modelo "Cadeado" (Lock-Modify-Unlock)

- É a abordagem pessimista.
- O desenvolvedor "A" avisa ao servidor: "Vou editar o index.html". O servidor coloca um cadeado no arquivo. Se o desenvolvedor "B" tentar editar, o servidor diz "Acesso negado, espere o 'A' terminar".
- E se o desenvolvedor "A" for almoçar, for demitido ou esquecer de tirar o cadeado? A equipe inteira fica travada.
- O SVN usava muito isso para arquivos que não podem ser mesclados, como imagens, PDFs ou planilhas do Excel.

A Era Centralizada (CVS e SVN)

O Modelo "Cópia e Mesclagem" (Copy-Modify-Merge)

- Esse é o padrão do SVN para código-fonte.
- A Regra do "Quem chega primeiro, ganha":
 - 1. O João e a Maria fazem o update do index.html ao mesmo tempo e começam a editar em suas máquinas.
 - 2. O João termina rápido e faz o commit no servidor. O servidor aceita, pois era o primeiro da fila.
 - 3. A Maria termina logo depois e tenta fazer o commit.
 - 4. O que o servidor faz? Ele rejeita o commit da Maria e lança um erro famoso chamado "Out of Date" (Desatualizado).
 - 5. A Solução: O servidor obriga a Maria a fazer um novo update (baixar o código do João). O próprio SVN tenta juntar as linhas (se foram em lugares diferentes). Se eles mexeram na mesma linha, dá conflito na máquina da Maria. Ela resolve, salva e, só então, o servidor aceita o novo commit dela.

A Era Centralizada (CVS e SVN)

Por que o SVN virou um problema para equipes gigantes?

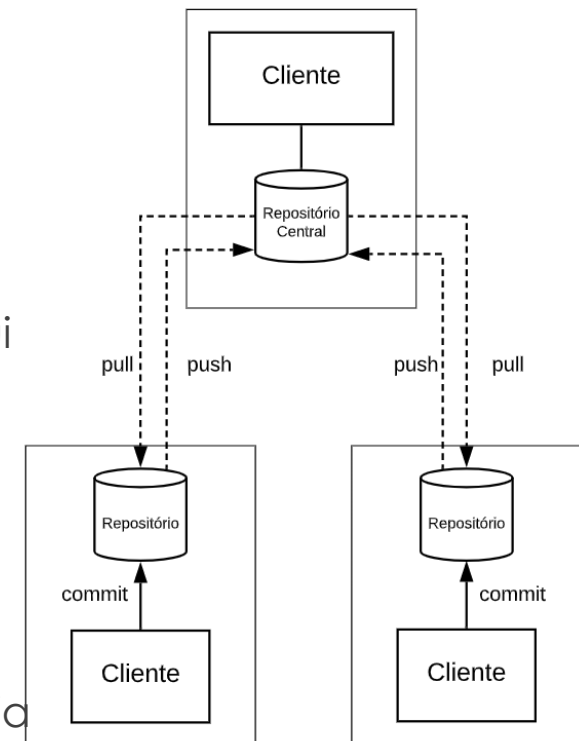
A Fila do Gargalo Central: Com 500 programadores enviando código ao mesmo tempo, o servidor central não dava conta de processar. Virava uma "fila de banco" virtual. Se o servidor caísse, o trabalho da empresa inteira parava.

O Peso das Branches: No SVN, criar uma "branch" (uma cópia do código para você trabalhar isolado) significava, literalmente, que o servidor tinha que copiar a pasta inteira do projeto em seus discos. Em projetos imensos de gigabytes, isso era extremamente lento.

O Medo do "Merge Hell" (Inferno do Merge): Como a integração no SVN era muito dolorosa e lenta, os programadores ficavam semanas trabalhando isolados por preguiça de enviar para o servidor. Quando finalmente tentavam juntar (merge) o trabalho de todo mundo, havia milhares de conflitos impossíveis de resolver, travando o lançamento do software.

A Revolução Distribuída (DVCS)

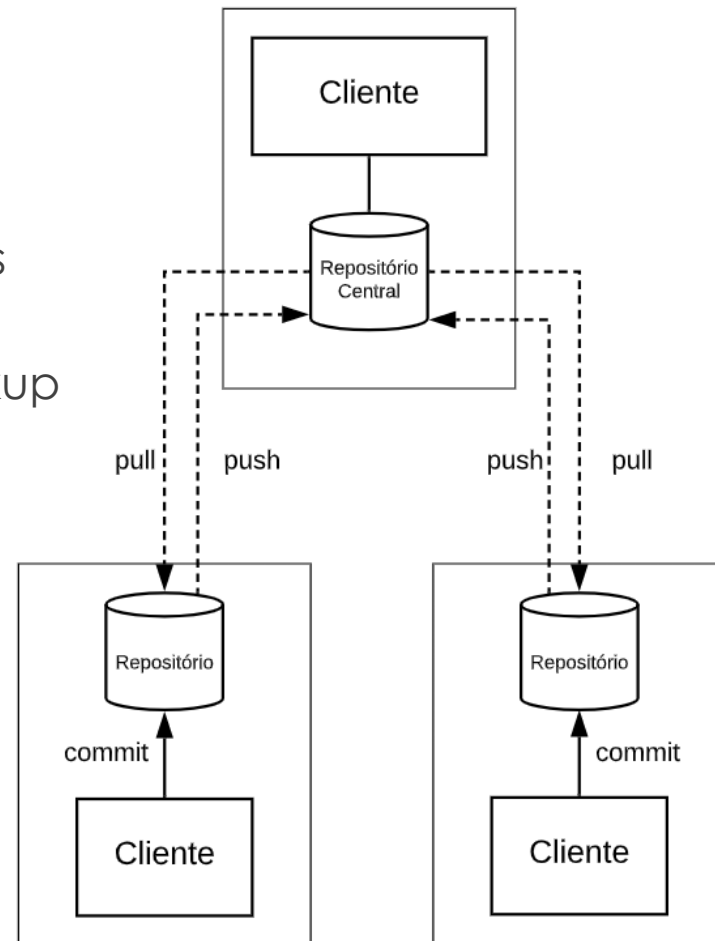
- A partir de 2005, começaram a surgir **Sistemas de Controle de Versões Distribuídos (DVCS)**, onde o poder vai para a máquina do desenvolvedor (a partir de 2005).
- Em vez de uma arquitetura cliente/servidor, um DVCS adota uma arquitetura peer-to-peer.
- Na prática, isso significa que cada desenvolvedor possui em sua máquina um servidor completo de controle de versões, que pode se comunicar com os servidores de outras máquinas.
- Em teoria, os clientes (ou peers) são funcionalmente equivalentes. Porém, na prática, costuma existir uma máquina principal, que armazena a versão de referência do código fonte.



A Revolução Distribuída (DVCS)

➤ As Super Vantagens:

- Offline: Dá para trabalhar e fazer commits no avião, sem internet.
- Velocidade: Commits são instantâneos porque rodam no seu HD local.
- Segurança: Todo mundo tem um backup completo da história do projeto.



Git



- A criação por Linus Torvalds.
- O Linux usava um DVCS pago chamado BitKeeper.
- Em 2005, a empresa do BitKeeper revogou as licenças gratuitas do pessoal do Linux.



Git



- A criação por Linus Torvalds.
- O Linux usava um DVCS pago chamado BitKeeper.
- Em 2005, a empresa do BitKeeper revogou as licenças gratuitas do pessoal do Linux.
- Linus Torvalds ficou furioso e, em poucos dias, criou o próprio sistema: o **Git** (código aberto e de linha de comando)



O motor x A oficina.



➤ GitHub é um serviço de hospedagem de código que usa o sistema Git para prover controle de versões.

Git: É o sistema de controle de versões (linha de comando, mora na sua máquina).

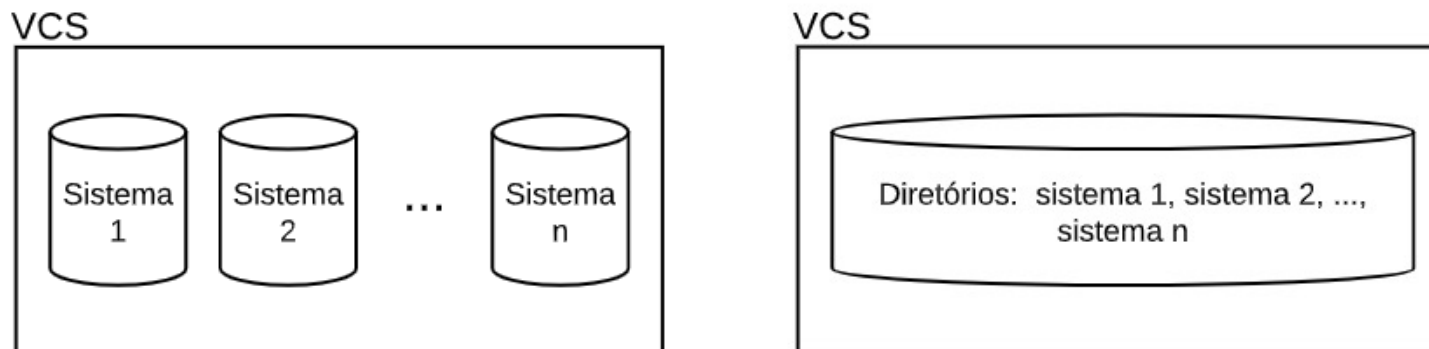
GitHub: É o serviço de hospedagem na web que usa o Git. Oferece repositórios para projetos abertos e privados.

Multirepos vs Monorepos

- Um VCS gerencia repositórios.
 - Assim, uma organização precisa decidir os repositórios que vai criar em seu VCS.

Multirepos : Um repositório separado para cada sistema. (Mais comum).

Monorepo (Google, Facebook, Microsoft): Todo o código da empresa inteira em um único repositório gigante.



Multirepos vs Monorepos

- Dentre as vantagens de monorepos podemos citar:
 - Não há dúvida sobre qual repositório possui a versão mais atualizada de um arquivo.
 - Incentivam o reuso e compartilhamento de código
 - Mudanças são sempre atômicas.
 - Facilita a execução de refactorings em larga escala.
- Por outro lado, monorepos requerem ferramentas para navegar em grandes bases de código.
 - Cada desenvolvedor terá em seu repositório local todos os arquivos de todos os sistemas da organização.
 - Exige ferramentas e plugins super pesados para a máquina aguentar ler tudo (como o Google precisou criar para o Eclipse).